

BASH SCRIPTING FUNDAMENTALS

Text Processing

grep, sed, awk, cut, sort, and practical pipelines

Merit Advisory

Bash Scripting Fundamentals Bootcamp

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: grep Essentials — 8 topics
- 2 Section 2: grep Patterns — 8 topics
- 3 Section 3: Selecting and Translating Text — 8 topics
- 4 Section 4: Sorting and Deduplicating — 8 topics
- 5 Section 5: Combining and Formatting Lines — 8 topics
- 6 Section 6: sed Basics — 8 topics
- 7 Section 7: awk Foundations — 8 topics
- 8 Section 8: awk Practice — 8 topics

- 9 Section 9: Practical Pipelines — 8 topics
- 1 Section 10: Watching and Boundaries — 8 topics
- 0
- 1 Section 11: Mistakes and Checkpoints — 8 topics
- 1

1 Section 1

grep Essentials

1.1 grep finds matching lines

- grep searches text and prints lines that match.

Search a file

```
grep ERROR app.log
```

- The pattern comes before the file names.

Search stdin

```
printf 'ok\nerror\n' | grep error
```

1.2 grep -n prints line numbers

- -n prefixes each match with its line number.

Line numbers

```
grep -n TODO script.sh
```

- Line numbers help you jump back into files.

Numbered stdin

```
printf 'a\nb\n' | grep -n b
```

1.3 grep -i ignores case

- -i matches uppercase and lowercase forms.

Case-insensitive log search

```
grep -i error app.log
```

- It is useful when input capitalization is inconsistent.

Case-insensitive word

```
printf 'Root\nuser\n' | grep -i root
```

1.4 grep -v inverts matches

- -v prints lines that do not match.

Drop comments

```
grep -v '^#' config.ini
```

- It removes noise from a stream.

Drop blank marker

```
printf 'ok\nskip\n' | grep -v skip
```


1.5 grep -w matches words

- -w requires the match to be a whole word.

Whole word user

```
grep -w user accounts.txt
```

- It avoids matching substrings inside longer names.

Avoid substring

```
printf 'cat\nscatter\n' | grep -w cat
```

1.6 grep -c counts matching lines

- -c prints the number of matching lines.

Count errors

```
grep -c ERROR app.log
```

- It counts lines, not individual matches.

Count stdin matches

```
printf 'x\ny\nx\n' | grep -c x
```

1.7 grep -l lists matching files

- -l prints file names instead of matching lines.

Files with TODO

```
grep -l TODO *.sh
```

- It stops after the first match in each file.

Configs with port

```
grep -l 'port=' configs/*
```

1.8 grep -R searches recursively

- -r or -R searches through directories.

Search scripts

```
grep -R TODO scripts/
```

- Recursive search can be noisy in large trees.

Search current tree

```
grep -R 'set -e' .
```

2 Section 2

grep Patterns

2.1 grep -E enables extended regular expressions

- -E supports clearer alternation and grouping syntax.

Alternation

```
grep -E 'ERROR|WARN' app.log
```

- It is the same family as egrep on many systems.

Grouped suffix

```
grep -E 'file\.(txt|csv)$' list.txt
```

2.2 grep -F searches fixed strings

- -F treats the pattern as literal text.

Literal dots

```
grep -F '10.0.0.1' access.log
```

- It avoids regex meaning for characters like . and *.

Literal brackets

```
grep -F '[prod]' app.log
```

2.3 grep context shows nearby lines

- -A prints lines after a match.

After context

```
grep -A 2 ERROR app.log
```

- -B prints lines before a match.

Both sides

```
grep -C 3 timeout app.log
```


2.4 Anchors match line positions

- `^` matches the start of a line.

Starts with error

```
grep '^ERROR' app.log
```

- `$` matches the end of a line.

Ends with done

```
grep 'done$' tasks.txt
```

2.5 Character classes describe sets

- [0-9] matches one digit in many grep modes.

Any digit

```
grep '[0-9]' ids.txt
```

- [[:digit:]] is a named character class.

Named class

```
grep '[[:digit:]]' ids.txt
```

2.6 Dot and star can overmatch

- . matches almost any single character.

Broad match

```
grep 'user=.*' config.ini
```

- * repeats the previous pattern zero or more times.

Specific match

```
grep 'user=[^ ]*' config.ini
```

2.7 Quote grep patterns

- Shell globbing can change unquoted patterns.

Quoted regex

```
grep '^[A-Z]' names.txt
```

- Single quotes preserve regex characters for grep.

Variable pattern

```
pat='ERROR'  
grep "$pat" app.log
```

2.8 ripgrep is a fast optional searcher

- rg is a modern recursive search tool.

Search with rg

```
rg TODO scripts/
```

- It respects many ignore files by default.

Line numbers

```
rg -n 'set -e' scripts/
```

3 Section 3

Selecting and Translating Text

3.1 cut -d -f selects delimited fields

- -d chooses the field delimiter.

First colon field

```
cut -d: -f1 /etc/passwd
```

- -f chooses one or more fields.

Second CSV field

```
cut -d, -f2 users.csv
```

3.2 cut -c selects character positions

- -c extracts by character position.

First eight chars

```
cut -c1-8 dates.txt
```

- Ranges like 1-8 select spans.

Single character

```
printf 'abc\n' | cut -c2
```


3.3 tr -d deletes characters

- tr reads stdin and writes translated stdout.

Remove carriage returns

```
tr -d '\r' < win.txt > unix.txt
```

- -d removes listed characters.

Remove digits

```
printf 'a1b2\n' | tr -d '0-9'
```

3.4 tr -s squeezes repeated characters

- -s replaces repeated listed characters with one.

Squeeze spaces

```
printf 'a  b\n' | tr -s ' ' '
```

- It is common for normalizing spaces.

Squeeze blank lines

```
tr -s '\n' < text.txt
```

3.5 tr translates characters

- tr maps characters from one set to another.

Uppercase

```
printf 'abc\n' | tr '[:lower:]' '[:upper:]'
```

- The first set and second set are positional.

Commas to tabs

```
tr ',' '\t' < data.csv
```

3.6 wc counts lines, words, and bytes

- -l counts lines.

Count lines

```
wc -l app.log
```

- -w counts words.

Count all three

```
wc -l -w -c README.md
```

3.7 head shows the beginning

- head prints the first lines of input.

First ten lines

```
head app.log
```

- -n controls how many lines to show.

First three lines

```
head -n 3 app.log
```

3.8 tail shows the end

- tail prints the last lines of input.

Last ten lines

```
tail app.log
```

- -n controls how many lines to show.

Last five lines

```
tail -n 5 app.log
```

4 Section 4

Sorting and Deduplicating

4.1 sort orders lines

- sort reads lines and writes them in order.

Sort names

```
sort names.txt
```

- Default sort order is lexicographic.

Sort stdin

```
printf 'b\na\n' | sort
```


4.2 sort -n orders numbers

- -n compares numeric values.

Numeric order

```
printf '10\n2\n' | sort -n
```

- Without -n, 10 can sort before 2.

Sort counts

```
sort -n counts.txt
```

4.3 sort -r reverses order

- -r reverses the final sort order.

Reverse text

```
sort -r names.txt
```

- Combine it with -n for largest numbers first.

Largest first

```
sort -nr counts.txt
```

4.4 sort -k chooses a key

- -k selects which field to sort by.

Sort by second field

```
sort -k2 users.txt
```

- Field splitting defaults to runs of blanks.

Sort CSV by third field

```
sort -t, -k3 users.csv
```

4.5 sort -u sorts unique lines

- -u removes duplicate lines while sorting.

Unique sorted names

```
sort -u names.txt
```

- It is a compact sort plus unique operation.

Unique from stdin

```
printf 'b\na\nb\n' | sort -u
```

4.6 uniq removes adjacent duplicates

- `uniq` compares neighboring lines only.

Adjacent duplicates

```
uniq sorted.txt
```

- Sort first when duplicates may be separated.

Sort then `uniq`

```
sort names.txt | uniq
```

4.7 uniq -c counts duplicate runs

- -c prefixes each output line with a count.

Count names

```
sort names.txt | uniq -c
```

- It counts adjacent runs, so sorted input matters.

Count statuses

```
cut -d' ' -f1 app.log | sort | uniq -c
```

4.8 uniq -d prints duplicated lines

- -d prints only lines that repeat in adjacent runs.

Find duplicate names

```
sort names.txt | uniq -d
```

- Sort first to find duplicates anywhere in a file.

Duplicate IDs

```
cut -d, -f1 users.csv | sort | uniq -d
```

5 Section 5

Combining and Formatting Lines

5.1 paste combines lines side by side

- paste joins corresponding lines from files.

Paste two files

```
paste names.txt scores.txt
```

- The default delimiter is a tab.

Comma delimiter

```
paste -d, names.txt scores.txt
```

5.2 join merges sorted files on a key

- join combines lines that share a field.

Join by first field

```
join users.sorted scores.sorted
```

- Input files must be sorted on the join field.

Join CSV-like data

```
join -t, users.csv scores.csv
```

5.3 column -t aligns tables

- column -t formats columns into a readable table.

Align whitespace table

```
column -t report.txt
```

- It is for display, not reliable data storage.

Align CSV display

```
column -s, -t users.csv
```

5.4 rev reverses characters on each line

- rev reverses each line independently.

Reverse text

```
printf 'abc\n' | rev
```

- It can help with suffix-oriented tricks.

Look at suffixes

```
rev files.txt | sort | rev
```

5.5 fmt refills paragraphs

- `fmt` wraps text into paragraphs.

Wrap prose

```
fmt notes.txt
```

- It is useful for plain prose.

Set width

```
fmt -w 60 notes.txt
```

5.6 fold wraps long lines

- fold breaks long lines at a width.

Fold at 40

```
fold -w 40 long.txt
```

- -w chooses the output width.

Fold on spaces

```
fold -s -w 72 paragraph.txt
```

5.7 expand converts tabs to spaces

- expand replaces tab characters with spaces.

Default expand

```
expand table.txt
```

- -t controls tab stops.

Four-space tabs

```
expand -t 4 Makefile
```

5.8 unexpand converts spaces to tabs

- unexpand replaces runs of spaces with tabs.

Leading blanks

```
unexpand indented.txt
```

- -a can convert beyond leading blanks.

All blanks

```
unexpand -a aligned.txt
```


6 Section 6

sed Basics

6.1 sed s replaces the first match

- s/pattern/replacement/ substitutes text.

Replace first match

```
sed 's/error/ERROR/' app.log
```

- By default, sed replaces the first match on each line.

Replace delimiter text

```
printf 'a:b\n' | sed 's/:/,/'
```

6.2 sed s with g replaces all matches

- The g flag means global within each line.

Replace all spaces

```
sed 's/ /_/g' names.txt
```

- Use it when repeated matches on a line should change.

Replace all commas

```
printf 'a,b,c\n' | sed 's/,/|/g'
```

6.3 sed -n suppresses automatic printing

- sed normally prints every processed line.

Print matching lines

```
sed -n '/ERROR/p' app.log
```

- -n stops that automatic output.

Print line two

```
sed -n '2p' file.txt
```

6.4 sed can print ranges

- Address ranges select a span of lines.

Lines two to five

```
sed -n '2,5p' file.txt
```

- A range can use line numbers or patterns.

Between markers

```
sed -n '/BEGIN/,/END/p' file.txt
```

6.5 sed d deletes selected lines

- The d command removes matching lines from output.

Delete blanks

```
sed '/^$/d' notes.txt
```

- It does not modify the original file by default.

Delete comments

```
sed '/^#/d' config.ini
```

6.6 sed line addresses target positions

- A number selects one line.

Delete first line

```
sed '1d' data.csv
```

- \$ selects the last line.

Print last line

```
sed -n '$p' data.csv
```

6.7 sed -i edits files in place

- -i changes the file instead of only stdout.

Preview change

```
sed 's/dev/prod/' app.conf
```

- Test without -i before modifying important files.

In-place with backup

```
sed -i.bak 's/dev/prod/' app.conf
```


6.8 sed can use alternate delimiters

- The delimiter after s does not have to be slash.

Path replacement

```
sed 's#/tmp#/var/tmp#g' paths.txt
```

- Alternate delimiters reduce escaping in paths and URLs.

URL replacement

```
sed 's|http://|https://|g' urls.txt
```

7 Section 7

awk Foundations

7.1 awk fields use \$1, \$2, and more

- awk splits each input line into fields.

Print first field

```
awk '{print $1}' users.txt
```

- \$1 is the first field and \$2 is the second field.

Print whole line

```
awk '{print $0}' users.txt
```

7.2 awk -F sets the field separator

- -F chooses how awk splits fields.

Colon field

```
awk -F: '{print $1}' /etc/passwd
```

- Use -F: for colon files and -F, for simple CSV.

CSV field

```
awk -F, '{print $2}' users.csv
```

7.3 awk NR counts records

- NR is the current input record number.

Number lines

```
awk '{print NR, $0}' file.txt
```

- For normal text, a record is usually a line.

Skip header

```
awk 'NR>1 {print $0}' data.csv
```

7.4 awk NF counts fields

- NF is the number of fields on the current record.

Print field count

```
awk '{print NF, $0}' file.txt
```

- \$NF is the last field.

Print last field

```
awk '{print $NF}' file.txt
```

7.5 awk patterns select records

- A pattern before braces decides whether the action runs.

Regex pattern

```
awk '/ERROR/ {print $0}' app.log
```

- Regex patterns use slashes.

Numeric pattern

```
awk '$3 > 80 {print $1}' scores.txt
```

7.6 awk BEGIN runs before input

- BEGIN actions run before the first record.

Print header

```
awk 'BEGIN {print "name score"} {print $1, $2}' scores.txt
```

- Use BEGIN to print headers or set variables.

Set separator

```
awk 'BEGIN {FS=":"} {print $1}' /etc/passwd
```


7.7 awk END runs after input

- END actions run after all records are read.

Count records

```
awk 'END {print NR}' app.log
```

- Use END for totals and summaries.

Print final total

```
awk '{sum+=$1} END {print sum}' nums.txt
```

7.8 awk print builds output records

- print joins arguments with the output field separator.

Print two fields

```
awk '{print $1, $3}' users.txt
```

- Commas in print are not literal commas.

Comma output

```
awk 'BEGIN {OFS=","} {print $1,$3}' users.txt
```

8 Section 8

awk Practice

8.1 awk can sum a column

- Add field values to a variable for each record.

Sum first column

```
awk '{sum += $1} END {print sum}' nums.txt
```

- Print the accumulated value in END.

Sum CSV amount

```
awk -F, '{sum += $3} END {print sum}' sales.csv
```

8.2 awk can compute averages

- Keep a running sum and count.

Average first column

```
awk '{sum+=$1} END {print sum/NR}' nums.txt
```

- Use END after all input has been processed.

Average score

```
awk 'NR>1 {sum+=$2; n++} END {print sum/n}' scores.txt
```

8.3 awk can count matching records

- Increment a variable only on selected records.

Count errors

```
awk '/ERROR/ {n++} END {print n}' app.log
```

- A regex pattern is enough for simple log counts.

Count status code

```
awk '$9 == 500 {n++} END {print n}' access.log
```

8.4 awk extracts and rearranges columns

- Print fields in the order you need.

Swap fields

```
awk '{print $2, $1}' names.txt
```

- Set FS for input and OFS for output.

CSV subset

```
awk -F, 'BEGIN{OFS=","} {print $3,$1}' users.csv
```

8.5 awk arrays group counts

- Associative arrays use strings as keys.

Count first field

```
awk '{count[$1]++} END {for (k in count) print k, count[k]}' data.txt
```

- Increment an array element for each record's group.

Count CSV status

```
awk -F, '{count[$2]++} END {for (k in count) print k,count[k]}' events.csv
```


8.6 awk conditions filter records

- Comparisons can use numbers or strings.

High scores

```
awk '$2 >= 90 {print $1}' scores.txt
```

- Logical operators combine conditions.

Two checks

```
awk '$3 == "prod" && $4 > 0 {print $1}' services.txt
```

8.7 awk split breaks a field apart

- `split(string, array, separator)` divides text.

Split date

```
awk '{split($1,d,"-"); print d[1]}' dates.txt
```

- The pieces are stored starting at index 1.

Split email

```
awk -F, '{split($2,a,"@"); print a[1]}' users.csv
```

8.8 awk -v passes shell values safely

- -v sets an awk variable before processing begins.

Pass threshold

```
limit=80  
awk -v limit="$limit" '$2 > limit {print $1}' scores.txt
```

- It avoids fragile quote mixing inside the program.

Pass name

```
name=alice  
awk -v name="$name" '$1 == name {print $0}' users.txt
```

9 Section 9

Practical Pipelines

9.1 Combine grep, cut, sort, and uniq

- Filter first to reduce the data stream.

Count error codes

```
grep ERROR app.log | cut -d' ' -f3 | sort | uniq -c
```

- Extract only the field you need.

Count users

```
cut -d, -f1 users.csv | sort | uniq -c
```

9.2 Log parsing starts with stable fields

- Know the log format before choosing field numbers.

Sample logs

```
head -n 5 access.log
```

- Use grep to narrow lines and awk to select fields.

Extract status

```
awk '{print $9}' access.log | sort | uniq -c
```

9.3 Counting errors is a common pipeline

- Choose the error marker precisely.

Simple error count

```
grep -c ERROR app.log
```

- Count matching lines with `grep -c` for simple cases.

Error categories

```
grep ERROR app.log | awk '{print $3}' | sort | uniq -c
```

9.4 Extract columns after confirming delimiters

- cut is best for simple single-character delimiters.

Simple CSV column

```
cut -d, -f2 users.csv
```

- awk is better for whitespace and rearranging fields.

Whitespace column

```
awk '{print $2}' table.txt
```


9.5 Top values need counts and numeric sort

- Count values with sort and uniq -c.

Top paths

```
awk '{print $7}' access.log | sort | uniq -c | sort -nr | head
```

- Sort counts numerically in reverse order.

Top users

```
cut -d, -f1 events.csv | sort | uniq -c | sort -nr | head
```

9.6 Whitespace cleanup normalizes input

- Trim or squeeze spaces before field extraction when needed.

Squeeze spaces

```
tr -s ' ' < messy.txt
```

- tr -s handles repeated single characters.

Trim trailing spaces

```
sed 's/[[:space:]]\+$//' messy.txt
```

9.7 dos2unix removes Windows line endings

- Windows text may end lines with carriage return plus newline.

Remove carriage returns

```
tr -d '\r' < win.txt > unix.txt
```

- The carriage return can break comparisons and scripts.

Detect carriage returns

```
grep '$\r' win.txt
```

9.8 Encoding affects text tools

- Text tools operate on bytes and characters according to locale.

Show locale

```
locale
```

- Unexpected encodings can break sorting and matching.

Byte-oriented sort

```
LC_ALL=C sort names.txt
```

10

Section 10

Watching and Boundaries

10.1 tail -f follows growing files

- -f keeps reading as new lines are appended.

Follow a log

```
tail -f app.log
```

- It is useful for live log watching.

Follow errors

```
tail -f app.log | grep ERROR
```

10.2 head and tail sample safely

- head checks the beginning of a large file.

Check shape

```
head -n 3 data.csv
```

- tail checks recent or ending lines.

Check ending

```
tail -n 3 data.csv
```

10.3 Words and substrings differ

- A substring match can find text inside longer values.

Substring

```
printf 'cat\nscatter\n' | grep cat
```

- `grep -w` limits matches to words.

Whole word

```
printf 'cat\nscatter\n' | grep -w cat
```


10.4 Locale changes sort order

- sort follows locale collation rules by default.

Default sort

```
sort names.txt
```

- LC_ALL=C gives byte-oriented order.

C locale sort

```
LC_ALL=C sort names.txt
```

10.5 Tabs and spaces are different delimiters

- A visual gap may be spaces, tabs, or both.

Tab fields

```
cut -f1 table.tsv
```

- cut -f defaults to tab-delimited fields.

Show tabs visibly

```
sed -n '1' table.tsv
```

10.6 Simple CSV is not all CSV

- `cut -d,` works only for simple comma-separated data.

Simple field

```
cut -d, -f3 users.csv
```

- Quoted commas require a real CSV parser.

Awk simple CSV

```
awk -F, '{print $3}' users.csv
```

10.7 Text tools expect text

- Binary input can produce strange terminal output.

Identify file type

```
file archive.bin
```

- grep may report binary file matches.

Extract printable text

```
strings archive.bin | head
```

10.8 Choose the smallest clear tool

- grep filters lines.

Line filter

```
grep ERROR app.log
```

- cut extracts simple fields.

Field summary

```
awk '{count[$1]++} END {for (k in count) print k,count[k]}' data.txt
```

11

Section 11

Mistakes and Checkpoints

11.1 Common grep mistake: unquoted patterns

- Unquoted patterns can be expanded by the shell.

Risky pattern

```
grep *.log files.txt
```

- A pattern that works in one directory may fail in another.

Quoted pattern

```
grep '*.log' files.txt
```

11.2 Common cut mistake: wrong delimiter

- cut does not guess delimiters.

Inspect sample

```
head -n 1 data.txt
```

- A comma delimiter will not split tab-separated data.

Choose delimiter

```
cut -d: -f1 /etc/passwd
```


11.3 Common uniq mistake: unsorted input

- `uniq` only removes adjacent duplicates.

Wrong for separated duplicates

```
uniq names.txt
```

- Separated duplicates remain unless input is sorted.

Global unique lines

```
sort names.txt | uniq
```

11.4 Common sed mistake: editing too soon

- sed -i changes files immediately.

Preview first

```
sed 's/old/new/g' file.txt
```

- Preview substitutions on stdout first.

Then edit with backup

```
sed -i.bak 's/old/new/g' file.txt
```

11.5 Common awk mistake: shell quoting

- awk programs often contain \$ characters.

Protect fields

```
awk '{print $1}' users.txt
```

- Single quotes protect \$1 from shell expansion.

Pass a value

```
field=2  
awk -v f="$field" '{print $f}' users.txt
```

11.6 Practice pipelines one stage at a time

- Run the first command and inspect its output.

Start simple

```
grep ERROR app.log
```

- Add one stage only after the previous stage is correct.

Add stages

```
grep ERROR app.log | awk '{print $3}' | sort | uniq -c
```

11.7 Review text processing choices

- Use grep for lines, cut for simple columns, sed for edits.

Line and column

```
grep active users.csv | cut -d, -f1
```

- Use sort and uniq for ordering and counts.

Numeric summary

```
awk '{sum+=$2} END {print sum}' scores.txt
```

11.8 Module 06 checkpoint

- Count a category from a log file.

Count categories

```
awk '{print $3}' app.log | sort | uniq -c
```

- Extract and sort one column from delimited text.

Clean and extract

```
tr -s ' ' < table.txt | cut -d' ' -f2
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory

06 · Text Processing

